

Hollow House Institute

Execution-Time Governance in AI Systems

Governing What Happens When the Model Runs

Author: Amy

Hollow House Institute | Principal Researcher & Author

Date: May 2026

Developed through directed AI-assisted research. Full authorship and intellectual responsibility retained by
Amy.

Publisher: Hollow House Institute

Distribution: Zenodo | GitHub | LinkedIn

Suggested Citation: Amy. (2026). *Execution-Time Governance in AI Systems: Governing What Happens When the Model Runs*. Hollow House Institute. DOI: 10.5281/zenodo.XXXXXXX

Abstract

Almost all current AI governance operates at design time — rules about training data, model architecture, evaluation benchmarks, and documentation requirements that are satisfied before a system is deployed. This paper argues that this framing has produced a governance architecture with a fundamental structural gap: it has no systematic answer for what happens when the model runs. Execution-time governance — the set of mechanisms that observe, constrain, and create accountability for AI system behavior in production, on real inputs, in real time — is the missing layer. We define its four core components: runtime monitoring, runtime constraint enforcement, runtime attestation, and intervention and rollback. We survey the current technical landscape across each component, finding significant capability in statistical monitoring tools and output filtering, but near-total absence of cryptographic attestation infrastructure and standardized intervention governance. We identify and analyze five structural challenges — the opacity problem, the scale problem, the latency problem, the adversarial problem, and the jurisdiction problem — that explain why this layer is hard to build and why the market has not built it spontaneously. We examine what functional execution-time governance architecture would require for a high-risk AI system in a regulated domain, and we assess where current frameworks — the EU AI Act, the NIST AI RMF, and ISO 42001 — fall short in mandating it. We conclude with seven specific recommendations directed at AI developers, regulators, and standards bodies. The central argument is simple: design-time governance is necessary but nowhere near sufficient. We are governing AI systems the way we govern bridges — inspecting the blueprints and the materials, but not the bridge while people are crossing it. For systems that adapt, drift, and produce billions of consequential outputs a day, that is not governance. It is documentation.

Author's Note on Co-Creation

This paper was developed through a directed research and drafting process in which I defined the conceptual framework, identified the core arguments, set the analytical boundaries, and determined what this paper would and would not claim. AI-assisted tools were used to support literature

synthesis, structural drafting, and document production under my ongoing direction. Every argument here reflects my intellectual position. I retain full authorship and responsibility for the ideas.

I'm documenting this process transparently because I think the field should. Using AI to accelerate serious research work is not a shortcut — it is a method, and like any method, what matters is who is driving it and whether the thinking is real. Mine is.

Table of Contents

1. A Problem That Governance Cannot Currently See

2. The Design-Time Bias in Current Governance

2.1 The EU AI Act: Thoroughness at the Gate

2.2 The NIST AI RMF: A Development-Phase Architecture

2.3 ISO 42001: Managing the System That Makes AI

2.4 Why Governance Frameworks Were Built This Way

3. What Execution-Time Governance Means

3.1 Runtime Monitoring

3.2 Runtime Constraint Enforcement

3.3 Runtime Attestation

3.4 Intervention and Rollback

4. The Technical Landscape: What Exists

4.1 Runtime Monitoring Platforms

4.2 Output Filtering and Constraint Enforcement

4.3 Runtime Attestation: The Near-Empty Category

4.4 Intervention Mechanisms

5. The Accountability Gap

6. What Makes Execution-Time Governance Hard

6.1 The Opacity Problem

6.2 The Scale Problem

6.3 The Latency Problem

6.4 The Adversarial Problem

6.5 The Jurisdiction Problem

7. What Functional Execution-Time Governance Looks Like

7.1 The Runtime Monitoring Specification

7.2 The Runtime Constraint Layer

7.3 Attestation Infrastructure

7.4 Intervention Protocol

7.5 The Continuous Improvement Loop

7.6 Architecture Summary

8. Policy Implications

8.1 What the EU AI Act's Article 72 Currently Requires

8.2 Minimum Viable Execution-Time Governance Requirements

8.3 Mandated Versus Recommended

9. Recommendations

10. Conclusion

References

1. A Problem That Governance Cannot Currently See

Consider an AI system deployed in a large public administration to support case routing decisions — determining which applicants are directed to which review pathways. Before deployment, the system passes every applicable pre-deployment evaluation. Its benchmark performance is strong across the test suite. Its technical documentation is complete. It satisfies the conformity assessment requirements under applicable regulation. The organization deploying it has completed its internal risk assessment. On paper, everything is in order. The system goes live.

Over the following six months, something happens that no one is watching for, because no one has the instruments to watch for it. The underlying input distribution shifts as the composition of the applicant pool changes seasonally. The system's confidence scores — already imperfectly calibrated — become progressively more miscalibrated under the new distribution. Outputs for applicants from specific linguistic communities, who were underrepresented in the evaluation data, begin to degrade in ways that are not visible from aggregate performance metrics. The system continues to operate within its documented operational envelope. It is not producing outputs that any individual reviewer would flag as obviously wrong. But it is, systematically, routing a particular group of applicants into less favorable pathways at a rate that would, if visible, be recognized as algorithmic discrimination. By the time a researcher notices a pattern in a downstream dataset, eighteen months and several hundred thousand decisions have passed. The design-time record tells you nothing about this. The deployment-time record barely exists.

Why does our governance architecture have no answer for this? The question is not rhetorical. There is a specific and tractable answer, and this paper makes it. The answer is that governance frameworks for AI systems have been built almost entirely around the period before deployment — the design and development phase in which the model is trained, evaluated, documented, and approved. The mechanisms that exist for the period after deployment are thin, inconsistent, and generally incapable of detecting or responding to behavioral drift in near-real-time. There is no layer of governance that travels with the model into production, observes what it actually does with real inputs, enforces defined constraints on its outputs, creates a verifiable audit trail of its behavior, and gives authorized actors the tools to intervene when something goes wrong.

We call this missing layer execution-time governance. This paper defines what it is, maps what currently exists against its four core components, analyzes why it is structurally underbuilt, describes what a functional architecture would look like in a high-risk deployment context, and identifies what regulatory frameworks would need to require in order to produce it. The goal is not to describe an ideal that is permanently out of reach. Governance frameworks routinely mandate technical infrastructure that did not exist before they mandated it. The goal is to be precise enough about what execution-time governance requires that developers, regulators, and standards bodies can make concrete decisions about building it.

2. The Design-Time Bias in Current Governance

Examining the three most significant AI governance frameworks that currently shape practice — the EU AI Act, the NIST AI Risk Management Framework, and ISO 42001 — reveals a consistent structural feature: the weight of their obligations falls before the system is deployed, and the obligations they impose after deployment are comparatively thin. This is not an accident of drafting. It reflects a deeper conceptual model that is, we will argue, substantially wrong for AI systems.

2.1 The EU AI Act: Thorough Before Deployment, Thin After

The EU AI Act (Regulation (EU) 2024/1689), which entered into force in August 2024, is the most comprehensive legislative attempt yet to govern AI systems at scale. For high-risk AI systems — those defined under Annex III as operating in consequential domains including employment, essential services, critical infrastructure, education, and justice — the pre-deployment obligations are demanding: quality management system, conformity assessment, extensive technical documentation, data governance requirements, specified accuracy and robustness thresholds, user transparency obligations, and registration in the EU database before the system touches a single real user.

All of these obligations apply to the period before deployment. The Act does include post-deployment obligations — most importantly the post-market monitoring system under Article 72, which requires providers to "actively and systematically collect, document and

analyse relevant data... on the performance of high-risk AI systems throughout their lifetime." Read carefully, though, Article 72 is a data collection and analysis obligation, not a runtime governance requirement. Its monitoring is oriented toward assessing whether the system continues to comply with the Act's design-time requirements. It does not specify continuous observation of outputs, near-real-time alerting on behavioral drift, cryptographic attestation of inference records, or defined intervention protocols. The monitoring plan is a document. The monitoring itself is a retrospective review. It is governance of the record, not governance of the behavior — and that distinction is the entire problem.

2.2 The NIST AI RMF: Built for Development, Not Deployment

The NIST AI Risk Management Framework, released in January 2023, organizes AI risk management around four functions: GOVERN, MAP, MEASURE, and MANAGE. The framework is voluntary, context-sensitive, and deliberately non-prescriptive — it is designed as a structure that organizations can adapt to their specific situations rather than a checklist. These are virtues, but they create a specific weakness in relation to execution-time governance.

The MAP function focuses on categorizing AI risks in context, which is primarily a pre-deployment analysis. The MEASURE function addresses evaluation and testing, which again centers on the development phase. The MANAGE function addresses how organizations respond to identified risks, but the framework's guidance on post-deployment monitoring is oriented toward the same periodic review model as the EU AI Act — checking whether the system continues to perform as expected, rather than continuously governing its behavior in production. NIST's parallel work on Special Publication 800-53, including the ongoing Control Overlays for Securing AI Systems (COSAiS) project launched in 2025, begins to address runtime security controls for AI systems, but this work is still developing and has not yet produced mandatory requirements. The AI RMF's discussion of "real-time monitoring" is brief and non-prescriptive — it appears in the context of AI safety as one practical approach among many, rather than as a structured governance requirement.

2.3 ISO 42001: Governing the Organization, Not the Inference

ISO/IEC 42001:2023 — the world's first international standard for AI management systems — specifies requirements for establishing, implementing, maintaining, and continually improving an AI management system within organizations. It follows the familiar Plan-Do-Check-Act structure, and like other management system standards, it focuses primarily on the organizational processes and governance structures surrounding AI development rather than on the technical behavior of AI systems in production. Certification to ISO 42001 demonstrates that an organization has a responsible management system for AI. It does not demonstrate — and does not claim to demonstrate — that the AI systems that organization deploys are continuously governed at the inference level. That is not a criticism of the standard. It is a description of its scope. But the scope is the problem.

2.4 The Product Safety Model and Why It Fails for AI

The design-time bias in these frameworks is not a failure of draftsmanship. It reflects a coherent — but ultimately inadequate — conceptual model: the product safety model. Medical devices, automobiles, industrial equipment, pharmaceuticals — the product categories that regulatory frameworks have governed most successfully over the past century share one critical feature. Their behavior is fixed at manufacture. A pacemaker does what it does regardless of who carries it. A car's braking system was designed and tested under defined conditions; it does not change how it brakes based on real-time inputs to a neural network. Pre-market evaluation makes sense for these products because that is where behavior is determined. Post-market surveillance catches manufacturing defects and long-term failure modes, but the core behavioral question — does this product do what it is certified to do? — was settled before it left the factory.

AI systems are categorically different. Their outputs are functions of runtime inputs, and those outputs can shift systematically without any change to the underlying model. A language model that performed well on its evaluation set will produce different output distributions on different input populations. A clinical decision support system trained on data from one hospital network may behave harmfully when deployed in a different patient population — not because the model changed, but because the world did. This is not a bug that better pre-deployment evaluation can eliminate. It is structural. Governance frameworks built on the product safety model are not merely incomplete for AI systems — they are a category error applied to the wrong kind of thing. And right now, that category error is our primary governance strategy.

Key Finding

The three most significant AI governance frameworks in current use — the EU AI Act, the NIST AI RMF, and ISO 42001 — place the preponderance of their compliance obligations at design and development time, before deployment. Their post-deployment requirements are oriented toward periodic review and retrospective analysis rather than continuous runtime governance. None currently specifies mandatory runtime monitoring thresholds, output constraint enforcement standards, cryptographic audit trail requirements, or defined intervention authorization protocols for high-risk AI systems in production.

3. What Execution-Time Governance Means

Before we can discuss what is missing, we need to be precise about what we mean. "Execution-time governance" is not a term of art that already exists in the governance literature. We introduce it to name a specific and coherent concept: the set of mechanisms that observe, constrain, and create accountability for AI system behavior as it occurs — in production, on real inputs, in real time. This is distinct from design-time governance (everything that happens before deployment), from post-hoc auditing (reviewing what happened after the fact), and from incident response (reacting to identified harms). Execution-time governance operates while the model is running.

Definition: Execution-Time Governance

Execution-time governance is the set of mechanisms that observe, constrain, and create accountability for AI system behavior as it occurs — in production, on real inputs, in real time. It encompasses four functional components: runtime monitoring, runtime constraint enforcement, runtime attestation, and intervention and rollback. Together, these components constitute a governance layer that operates alongside the model in production, as distinct from governance mechanisms that apply before deployment (design-time governance) or after harm has occurred (post-hoc audit and incident response).

Execution-time governance has four core components, each addressing a distinct governance function. They are cumulative, not alternative — a complete architecture requires all four, and the absence of any one creates a structural gap that the others cannot fill.

3.1 Runtime Monitoring

Runtime monitoring is the continuous observation of system outputs — and, where accessible, system internals — for signals that indicate behavioral drift, policy violations, or conditions that warrant human review. The key word is continuous. Runtime monitoring is not the same as logging data for later analysis, and it is not the same as periodic performance review. It requires defined monitoring specifications: what signals are observed, at what frequency, with what statistical methods, and against what alert thresholds. A runtime monitoring system that has not specified its alert thresholds is not a governance mechanism — it is a data collection exercise.

What signals are relevant? At minimum, runtime monitoring for a high-risk AI system should track: distributional shift in input characteristics (detecting when the system is receiving inputs that differ significantly from its evaluation population); output distribution drift (detecting when the statistical properties of outputs are changing over time); confidence calibration metrics (detecting when the system's confidence scores are becoming a less reliable predictor of actual accuracy); and performance disaggregation by subgroup, where feasible, to detect differential degradation across population segments. For systems producing natural language outputs, semantic monitoring — tracking shifts in the semantic properties of outputs — adds an additional layer. The point is that what is monitored must be specified in advance, and the thresholds that trigger alerts must be defined before the system goes live, not discovered after a problem has already caused harm.

3.2 Runtime Constraint Enforcement

Runtime constraint enforcement is the set of technical mechanisms that prevent certain outputs from being produced, regardless of what the underlying model would otherwise generate. This is the enforcement layer — the governance that does not merely observe what the model does but actively shapes what it can do. It includes output filters (mechanisms that evaluate proposed outputs against defined criteria and block or transform those that fail), confidence thresholds (mechanisms that prevent outputs below a defined confidence level from being surfaced to users without human review), domain restrictions (mechanisms that prevent the system from

operating outside its specified operational scope), and mandatory escalation triggers (mechanisms that route specific categories of outputs — those touching defined sensitive topics, for example — to human reviewers before they reach users).

The governance significance of the constraint layer is that it separates what the model can produce from what the system allows to be produced. This distinction matters for accountability: if a harmful output reaches a user, the question is not only what the model generated but whether the constraint layer was correctly specified and functioning. A properly governed system has a documented, versioned, auditable constraint specification that an external reviewer could inspect to determine whether the constraints in place were appropriate for the system's operational context. Most current deployments do not have this.

3.3 Runtime Attestation

Runtime attestation is the capacity to produce a verifiable record of what happened during a specific inference event: what input was received, what model state was operative, what the system produced, and what constraints were applied. This is the audit trail that makes accountability meaningful rather than aspirational. Without attestation, accountability claims in AI governance are largely rhetorical — a developer can assert that their system behaved correctly, a deployer can assert that their constraints were functioning, and a regulator can assert that monitoring was in place, but none of these assertions can be independently verified in relation to specific decisions that affected specific people.

Attestation is distinct from logging. Logging produces a record. Attestation produces a record that can be verified — cryptographically bound to the system state at the time of inference, tamper-evident, and accessible to authorized reviewers without requiring trust in the entity that created the log. The technical mechanisms for this kind of attestation exist in adjacent domains: trusted execution environments (TEEs) provide cryptographic attestation of the compute environment in which code runs, and confidential computing architectures extend this to protect data during inference. What does not yet exist at any meaningful scale is the governance framework that specifies what must be attested, how attestation records must be stored and retained, who has access to them, and how they can be used in a regulatory or legal proceeding.

3.4 Intervention and Rollback: Who Decides, and Under What Rules

Intervention and rollback addresses the governance of what happens when the monitoring system detects a problem: who can authorize what kinds of actions, under what conditions, with what documentation requirements, and subject to what review. At one end of the spectrum, intervention might mean rate-limiting or suspending a feature while an issue is investigated. At the other, it might mean immediately rolling back to a prior model version, restricting operation to a subset of domains, or taking the system offline entirely. The governance question is not merely technical — it is about the authorization chain. In a regulated high-risk deployment, the decision to roll back a model or restrict its operation is consequential: users depending on the system may be affected, and the action itself may need to be documented for regulatory purposes. An intervention protocol specifies who can authorize each level of intervention, what

information must be available to authorize it, what documentation is required, and what post-hoc review is triggered by an intervention event.

4. The Technical Landscape: What Exists

Against the four-component definition above, we can now survey what actually exists. The picture is uneven: some components have meaningful technical infrastructure, though it is rarely deployed in a genuinely governance-ready form. Others have almost nothing. The unevenness itself is informative — it reflects where market incentives have and have not been sufficient to produce governance-relevant infrastructure in the absence of regulatory mandates.

4.1 Runtime Monitoring: Capable Tools, No Governance Framework

The commercial MLOps ecosystem has produced a substantial set of runtime monitoring tools in recent years. MLflow, originally developed at Databricks, provides experiment tracking, model versioning, and basic production monitoring capabilities. Weights & Biases offers comprehensive logging and monitoring for model training and deployment. Arize AI is purpose-built for ML observability in production, offering statistical drift detection across feature distributions and output predictions, with specific capabilities for embedding-level drift monitoring relevant to large language model deployments. Fiddler AI combines monitoring with explainability features, including bias tracing across demographic subgroups and support for standard fairness metrics such as demographic parity and equalized odds — relevant for the kinds of differential performance degradation we described in our opening scenario. WhyLabs offers a statistically efficient monitoring approach based on compact profiling rather than storing every prediction, making it practical for high-throughput deployments.

These platforms are technically impressive and represent a genuine advance. But their limitations matter for governance. They monitor statistical properties of output and feature distributions — they detect that something has shifted, not necessarily what shifted or why. A drift metric is a starting point for a governance investigation, not an answer. Most do not monitor semantic content or policy compliance: they cannot determine whether outputs are within the normative constraints specified by the system's governance documentation, only whether they are statistically consistent with prior distributions. And they are almost entirely invisible to external auditors. A developer can run Arize AI or Fiddler AI in production and generate monitoring dashboards that demonstrate responsible practice internally — but that data is proprietary. There is no standard for what must be monitored, how results must be reported, or what access regulators or auditors have to the monitoring record. The infrastructure exists. The governance framework that would make it mean anything for accountability does not.

4.2 Output Filtering: Real but Proprietary and Brittle

Output filtering as a technical practice is relatively mature, at least for the major frontier AI developers. Anthropic's Constitutional AI approach, described in Bai et al. (2022) and

elaborated in subsequent work, uses a set of explicit principles to guide model training and output evaluation, creating a preference model trained to score outputs against normative criteria and then using that preference model in reinforcement learning to shift the base model toward constrained outputs. The result is a form of constraint enforcement that is baked into the model rather than applied as a post-hoc filter. Rule-based output classifiers — pattern-matching systems that evaluate outputs against defined prohibited content categories — represent an older and simpler approach, and they continue to be widely used alongside learned classifiers.

The limitations are well-understood and significant. Trained safety layers are brittle to adversarial prompting — inputs crafted to elicit policy-violating outputs can often bypass them, and this brittleness generalizes across domains in unpredictable ways. Coverage across languages and cultural contexts is uneven; safety layers trained primarily on English-language data frequently perform worse on other languages, which is exactly the failure mode that produces the differential degradation we described in the opening scenario. Most importantly: existing output constraint mechanisms are almost entirely opaque to parties outside the developer. An external auditor cannot inspect what constraints are in place, whether they are functioning as documented, or whether they are appropriate for the operational context. Nobody wants to say this plainly, but the constraint layer is currently treated as a proprietary implementation detail rather than as a governance artifact subject to external review. That is not governance. That is a vendor promise.

4.3 Runtime Attestation: Almost Nothing Exists

The most striking gap in the current technical landscape is runtime attestation — the capacity to produce cryptographically verifiable records of inference events that can support external accountability. Almost nothing exists at the industry level for this purpose. Trusted execution environments — hardware-enforced secure computation zones supported by Intel TDX, AMD SEV, ARM TrustZone, and NVIDIA's confidential computing features in its Hopper GPU architecture — provide attestation of the compute environment: a cryptographically signed assertion that a specific piece of code ran in a specific hardware environment without interference. Google's Confidential Neural Computing project has demonstrated that generative AI workloads can be run in TEEs with manageable performance overhead. Recent research on "Attestable Audits" (Schnabl et al., 2025) has demonstrated the technical feasibility of cryptographically attesting that a specific model version ran on a specific dataset in an auditable way, with performance overhead in their prototype running at approximately twice the cost of standard GPU inference for the CPU-constant variant.

What these technical advances do not yet provide is governance-ready attestation infrastructure for inference events. TEE attestation proves that a computation ran in a secure environment; it does not produce a semantically meaningful, interpretable record of the specific inference — what input was received, what model state was operative, what output was produced, what constraints were applied — in a form that would be useful to a regulatory auditor. Converting the technical capability into governance infrastructure requires standardized attestation schemas, defined retention and access policies, legal frameworks for the use of attestation records in regulatory proceedings, and organizational infrastructure to manage this across large-scale

deployments. None of these exist in any systematic form. This is the largest single gap in the current execution-time governance landscape, and it is one that neither the market nor any existing regulatory framework is actively closing.

Critical Gap: Runtime Attestation

There is currently no industry-level standard for runtime attestation of AI inference events. While trusted execution environments can cryptographically attest the compute environment, no standard exists for what must be recorded during an inference event, how that record must be structured, how it must be retained, or how it can be accessed for accountability purposes. This means that when an AI system causes harm, there is typically no tamper-evident record of what the system actually did — only whatever logging the developer chose to implement, in whatever format they chose, retained for whatever period they chose, and accessible only to the developer unless they choose to disclose it. This is not an audit trail. It is a diary entry.

4.4 Intervention Mechanisms

Technical mechanisms for intervening in the operation of a deployed AI system exist at the operator level and are widely used. Rate limiting, feature flags, A/B testing infrastructure, user-level access controls, and model version rollback capabilities are standard MLOps practice. What does not exist in any standardized form is the governance framework for how these capabilities are exercised in a regulated context. Who within an organization can authorize an emergency model rollback? What documentation is required to authorize an intervention that restricts system operation? What post-hoc review is triggered when an intervention occurs? What reporting obligations to regulators apply? These governance questions are not currently addressed by any major framework in a systematic way. Organizations that deploy high-risk AI systems are generally managing intervention decisions through the same organizational processes they would use for any software change — which means that the specific governance dimensions of AI system interventions (the downstream effects on affected users, the accountability implications of the decision, the regulatory reporting requirements) are often not systematically addressed.

5. The Accountability Gap

The absence of execution-time governance produces an accountability gap that becomes visible most clearly when something goes wrong. Current governance frameworks are reasonably good at assigning design-time accountability — the developer who deployed a system without adequate evaluation, the deployer who ignored documented limitations, the organization that failed to complete its conformity assessment. These forms of accountability are legible because they attach to documented decisions made at identifiable points in time. Runtime accountability is fundamentally more difficult, and current frameworks have, in practice, no coherent answer for it.

Consider three scenarios that governance frameworks are regularly asked to address. In the first, the model has not changed, but the input distribution has shifted. A clinical decision support system was evaluated on a patient population with a particular age distribution and trained principally on data from hospital networks serving urban centers. Over the eighteen months after deployment, the system is adopted by a rural hospital network serving a substantially different patient population. Performance degrades. Who is accountable? The developer documented the evaluation population. The deployer did not assess whether the system was appropriate for their patient population before adopting it. The current framework has partial answers — the deployer may have violated their obligations under Article 25 of the EU AI Act to implement the system appropriately for their context — but it has no mechanism for detecting that performance has degraded in the first place, because there is no runtime monitoring requirement that would surface this signal. The accountability framework has an answer in principle; it has no way to detect that the answer is needed.

In the second scenario, the output is technically within specifications but causes harm in an unanticipated context. An AI-assisted recruitment tool was evaluated against a test set derived from the company's historical hiring data and found to perform within acceptable accuracy bounds. In production, it is used in a context — reviewing applications from candidates in a jurisdiction with a specific labor market structure — that differs enough from the training distribution that its implicit scoring patterns produce systematically disadvantaged outcomes for a protected group. The output is not violating any configured rule; it is within the envelope defined by the system's operational parameters. No existing governance mechanism detects this pattern because pattern detection in output distributions across demographic subgroups requires active disaggregated monitoring that the framework did not mandate and the developer did not implement. By the time the pattern is discovered through external analysis, years have passed and thousands of decisions have been affected. The accountability question — was this a design-time failure (inadequate evaluation), a deployment-time failure (inadequate monitoring), or an inevitable artifact of context shift? — has no clean answer, and the absence of a runtime audit trail means it cannot be reconstructed precisely enough to give one.

The third scenario is the most legally complex. A human reviewer, using an AI system's recommendation, makes a consequential decision that causes harm. The AI system recommended one course of action; the human approved it. The recommendation was at the boundary of the system's reliable operating range — a condition that would have been flagged under a properly calibrated runtime confidence threshold, but no such threshold was implemented. Who is accountable? The human reviewer, for approving a recommendation they should have scrutinized more carefully? The deployer, for failing to implement appropriate human-in-the-loop safeguards? The developer, for not making the system's confidence calibration transparent enough for the deployer to configure appropriate thresholds? Current frameworks have fragments of answers to each of these questions, but they do not add up to a coherent accountability regime. They were not designed for a world where the failure mode is the silent, gradual degradation of a deployed system's behavior in the absence of any observable triggering event.

Key Finding

Current AI governance frameworks create legible design-time accountability — for documented decisions made before deployment. They cannot cleanly assign accountability for runtime behavioral failures: cases where the model has not changed but the context has; cases where outputs are technically within specifications but are systematically harmful in a specific deployment context; cases where human-in-the-loop processes have failed to catch problems that runtime governance would have surfaced. These failure modes are not edge cases. They are the characteristic failure modes of AI systems deployed at scale over time.

6. What Makes Execution-Time Governance Hard

The gap between the governance architecture we have and the one we need is not simply a matter of political will or regulatory urgency, though both are relevant. Execution-time governance is genuinely technically and organizationally difficult, in ways that are worth understanding precisely because misunderstanding them leads to governance proposals that are not actually implementable.

6.1 The Opacity Problem

Most production AI systems — particularly large neural networks deployed for natural language processing, image recognition, or decision support — are not interpretable at the level that meaningful runtime governance would require. You can observe outputs. You can measure output distributions. What you cannot do, with current technology, is inspect the internal computation that produced a specific output in a semantically meaningful way. This is not merely a practical limitation; it reflects deep properties of how large neural networks represent and process information. Interpretability research is active and has produced useful tools — feature attribution methods like SHAP values, attention visualization, probing classifiers — but none of these provides the kind of step-by-step, human-readable account of how an output was produced that would allow a governance inspector to verify that a specific inference was within the system's approved operational parameters.

What this means for execution-time governance is that the most useful forms of runtime constraint enforcement require some access to what is happening inside the computation. Output-level filtering catches specific prohibited content categories, but it cannot catch the subtler failure modes — confidence miscalibration, distributional shift-induced bias, context mismatch — without more information than is available at the output surface. Execution-time governance will therefore necessarily be statistical and probabilistic. It can identify patterns suggesting something has gone wrong. It cannot, in most cases, explain at the computation level why. Governance designers need to build for that reality rather than waiting for interpretability research to solve a problem it may not be able to solve.

6.2 The Scale Problem

Large-scale AI deployments handle millions of inferences per day. A language model API serving a significant user base may process tens or hundreds of millions of requests daily. Runtime governance cannot review each one; it must rely on sampling, statistical aggregation, and automated detection methods to identify patterns that warrant human review. But those automated detection methods are themselves systems that require governance. What is the sampling strategy? At what statistical threshold does an automated alert trigger? Who reviews alerts? How quickly must alerts be resolved? How are sampling strategies adapted as the input distribution changes? These are not simple operational questions — they are governance design questions that require careful specification and ongoing review. The scale problem does not make runtime governance impossible, but it means that the governance architecture itself is more complex than a simple monitoring requirement might suggest.

6.3 The Latency Problem

Many AI deployments operate under real-time or near-real-time constraints. A clinical decision support system surfacing information to a clinician during a consultation, a content moderation system reviewing user posts before they are published, a fraud detection system evaluating transactions — all of these have latency requirements that limit the computational complexity of runtime governance checks. A governance check that adds 200 milliseconds to every inference may be acceptable in some contexts and entirely impractical in others. This constraint is genuine and it shapes what kinds of runtime constraint enforcement are actually deployable. Complex semantic analysis of every output, even if technically possible, may not be feasible within latency constraints at production scale. This is not an argument against runtime constraint enforcement; it is an argument for a governance framework that is realistic about the tradeoffs involved in deploying it.

6.4 The Adversarial Problem

Runtime governance mechanisms become attack surfaces. If an adversary knows that a system filters outputs containing specific patterns, they can probe for inputs that produce harmful outputs without triggering those patterns — a well-documented class of attacks known as adversarial examples and jailbreaks. Conversely, if the monitoring system flags distributional anomalies, an adversary who understands the monitoring parameters can craft attacks that stay within the monitored distribution while still producing harmful outputs. This is a fundamental tension in security and in governance: any mechanism that enforces defined behavior also defines the space of behaviors that circumvent enforcement. The adversarial problem does not argue against runtime governance; it argues for designing runtime governance mechanisms with adversarial robustness as an explicit requirement, and for maintaining the confidentiality of specific detection thresholds and constraint parameters in security-sensitive deployments.

6.5 The Jurisdiction Problem

Modern AI system deployments are almost never jurisdictionally simple. The compute may run in cloud data centers in one jurisdiction. The model may have been trained by a developer headquartered in another. The operator deploying the model may be licensed in a third. The

users being served may be in a fourth, fifth, and sixth. Runtime governance — particularly runtime attestation and the access rights it implies for regulators and auditors — cuts across all of these jurisdictions simultaneously. An EU regulator seeking access to the runtime audit records of a system serving EU users may be requesting data that resides on servers in the United States, operated by a company incorporated in the Cayman Islands, using a model developed in the United Kingdom. The technical infrastructure for runtime attestation needs a jurisdictional framework to attach to, and that framework does not currently exist in any coherent form.

7. What Functional Execution-Time Governance Looks Like

Having defined what execution-time governance is, surveyed what exists, and analyzed why it is hard, we can now describe what functional execution-time governance architecture would actually require. We do this in the context of a high-risk AI system — a clinical decision support system providing recommendations to clinicians in a hospital setting — because this is a context where the stakes are high, the regulatory environment is demanding, and the governance architecture needed is therefore most fully developed. The architecture we describe is not a product specification. It is a governance specification: a description of the governance functions that would need to be fulfilled, and the minimum technical infrastructure required to fulfill them.

7.1 The Runtime Monitoring Specification

A runtime monitoring specification is a document, versioned and governed alongside the system's technical documentation, that defines precisely what is being monitored, how, and by whom. For our clinical decision support system, a minimum viable monitoring specification would include the following elements. Input monitoring: statistical testing at defined intervals — at minimum daily, ideally continuous — for distributional shift in patient demographics and clinical presentation inputs relative to the evaluation population, with alert thresholds specified using defined statistical tests (for example, Population Stability Index exceeding 0.2, or Kolmogorov-Smirnov test p-value below 0.05). Output monitoring: tracking of recommendation distribution across defined clinical categories, with alerting on deviations exceeding pre-specified bounds. Confidence calibration monitoring: Expected Calibration Error tracked against the baseline established at evaluation, with alerts triggered when ECE exceeds a defined threshold. Disaggregated performance monitoring: where ground truth feedback is available, performance metrics tracked separately by key demographic subgroups, with alerts triggered when subgroup performance diverges from aggregate performance by more than a defined margin.

The monitoring specification must also define alert response protocols — not just alert thresholds. An alert that sits in a queue and gets reviewed weekly by a junior engineer is not governance. Each alert category needs a defined response protocol: who is notified, within what timeframe, what initial investigation is required, what escalation path applies if a problem is confirmed, and what documentation is required throughout. The monitoring specification is a

governance document, not a technical configuration file. It should be subject to the same review and approval processes as the rest of the system's governance documentation.

7.2 The Runtime Constraint Layer

The runtime constraint layer is implemented as a policy enforcement point that sits between the model and its output interface. For our clinical decision support system, this layer would include confidence thresholds — any recommendation below a defined confidence level is surfaced to the clinician only with a mandatory uncertainty disclosure, and recommendations below a lower threshold require human review before surfacing. Domain scope enforcement: the constraint layer maintains a defined list of clinical domains within which the system is approved to operate, and routes queries that fall outside this scope to a human specialist rather than generating a model recommendation. Mandatory escalation triggers: defined clinical scenarios — certain patient presentations, certain drug interaction contexts — are routed to senior clinical review regardless of the model's confidence score. Population scope checks: where feasible, the constraint layer evaluates whether the input characteristics are within the bounds of the evaluation population, flagging potential out-of-distribution inputs.

The governance requirements for the constraint layer are as important as its technical design. The constraint specification must be documented, versioned, and approved through the same governance process as the model itself. Changes to the constraint layer — even those that appear to be improvements — must go through a documented change management process. The constraint layer must be auditable: an authorized reviewer should be able to inspect the current constraint specification and confirm that it matches the approved documentation. In the context of IEC 62443's security level framework, which applies to operational technology systems where AI is increasingly deployed, this kind of change management for control logic modification provides a useful partial analogy: model updates and constraint updates should be governed with the same rigor as changes to industrial control system programming.

7.3 Attestation Infrastructure

The attestation infrastructure is the component that is most technically underdeveloped relative to what governance would require. For our clinical decision support system, minimum viable attestation would require the following. At the inference level: a cryptographically signed record for each inference event, generated within the execution environment, that includes a hash of the model weights operative at the time of inference; a hash or representation of the input (subject to privacy constraints appropriate to clinical data); the system's output, including confidence scores; the constraint layer state and any constraints applied; and a timestamp. At the system level: a versioned and signed record of the constraint specification in operation during each time period. At the monitoring level: a tamper-evident log of monitoring alerts generated, responses initiated, and escalations triggered.

Recent technical work in confidential computing suggests the infrastructure for this kind of attestation is closer than it might appear. NVIDIA's Hopper architecture supports confidential GPU computing with performance overhead that is manageable for many production workloads. Research on attestable audits using TEEs has demonstrated the feasibility of producing

cryptographically verifiable records of model behavior, though with significant performance overhead in current implementations. What the technical capabilities lack is the governance framework they need to attach to: standardized attestation schemas, defined access protocols for authorized auditors, clear rules about retention periods and cross-border data access, and legal frameworks for the use of attestation records as evidence. Building the governance framework is more urgent than waiting for the technical capabilities to mature further.

7.4 Intervention Protocol

The intervention protocol defines the governance of runtime interventions: who can authorize what, under what conditions, with what documentation, and subject to what review. For our clinical decision support system, we propose a tiered intervention framework with three levels. Level 1 — Operational Adjustment: minor parameter adjustments within pre-specified ranges, authorized by the system operator, logged automatically, reviewed at next scheduled governance review. Level 2 — Constraint Modification: changes to the constraint layer specification, authorized by the designated AI governance officer and clinical lead, documented with rationale, requiring notification to relevant oversight bodies and post-hoc review within thirty days. Level 3 — Emergency Suspension or Rollback: suspension of the system or rollback to a prior model version, authorized by the clinical governance board with emergency authorization by the chief medical officer, requiring immediate notification to relevant regulatory bodies, and triggering a mandatory post-incident review with documented findings within ninety days.

The intervention protocol must also address the question of rollback capability. A system for which rollback to a prior model version is technically possible but organizationally undocumented is not meaningfully governed at the rollback level. The prior model version must exist and be accessible. The process for deploying it must be documented and tested. The organizational authority to authorize deployment must be clear. This sounds obvious, but in practice many organizations that deploy AI systems have not formally specified these things, treating rollback as an operational convenience rather than a governance requirement.

7.5 The Continuous Improvement Loop

Execution-time governance creates value beyond the governance function itself when the intelligence it generates feeds back into model evaluation and retraining governance. The monitoring system that detects drift in clinical decision support recommendations across a specific patient subgroup is producing exactly the kind of data that should inform the next evaluation cycle and, where appropriate, retraining decisions. But this feedback loop needs to be formal, documented, and governed — not an informal channel by which monitoring alerts prompt ad hoc model updates. A governed continuous improvement loop specifies how monitoring findings are reviewed at defined intervals, how they are assessed for significance, what criteria trigger a formal model review, what process governs retraining decisions, and how changes to the model are reflected in updated governance documentation. This is the mechanism by which execution-time governance connects to the design-time governance cycle, creating a genuinely iterative rather than one-shot governance architecture.

7.6 Architecture Summary

The table below provides a structured summary of the functional execution-time governance architecture described above, organized by component, with minimum viable requirements and relevant technical and standards references.

Component	Function	Minimum Viable Requirement	Technical Reference	Current State
Runtime Monitoring Specification	Define what is observed, at what frequency, with what alert thresholds	Documented, versioned, approved; covers input drift, output drift, calibration, disaggregated performance; alert response protocols defined	NIST SP 800-53 Rev.5 (SI-4, AU-2); COSAiS overlays (2025–2026)	Technical tools exist (Arize, Fiddler, WhyLabs); governance specification largely absent
Runtime Constraint Layer	Enforce behavioral limits on output regardless of model behavior	Documented constraint specification; versioned; auditable; change-managed ; includes confidence thresholds, domain scope, escalation triggers	Constitutional AI (Bai et al., 2022); IEC 62443 change management (partial analogy)	Filtering exists but is proprietary, invisible to auditors, inconsistently specified
Runtime Attestation	Produce verifiable records of inference events for accountability	Cryptographically signed inference records; model version hash; constraint state; retention policy; auditor access protocol	TEE attestation (Intel TDX, NVIDIA Hopper); Attestable Audits (Schnabl et al., 2025); Confidential Neural	Technical infrastructure emerging; governance framework absent

			Computing (Google, 2024)	
Intervention Protocol	Define who can authorize what interventions under what conditions	Tiered authorization chain; documentation requirements; post-hoc review triggers; rollback capability tested and documented; regulatory notification obligations	NIST SP 800-53 Rev.5 (IR family); IEC 62443-2-x (security management)	Technical mechanisms exist; governance framework not standardized
Continuous Improvement Loop	Feed runtime findings back into model evaluation and retraining governance	Formal review cycle; defined criteria for triggering model review; governed retraining process; updated governance documentation	NIST AI RMF IMPROVE function; EU AI Act Article 72 (partial)	Exists informally in some organizations; rarely formalized or governed

8. Policy Implications

The governance architecture described in Section 7 does not emerge spontaneously from market incentives. Building runtime monitoring infrastructure, maintaining attestation systems, and implementing formal intervention protocols all cost money. Without regulatory requirements that make execution-time governance a condition of deployment for high-risk AI systems, the market will systematically underinvest in it — not because developers are irresponsible, but because the costs are visible and the benefits (prevention of harms that may materialize only later, and may not be attributable to specific governance failures) are diffuse. What, then, would regulatory frameworks need to add?

8.1 What the EU AI Act's Article 72 Currently Requires vs. What It Would Need to Require

Article 72 requires providers to "actively and systematically collect, document and analyse relevant data... on the performance of high-risk AI systems throughout their lifetime." The implementing act that is supposed to lay down detailed provisions establishing a template for the post-market monitoring plan — a template that would specify what must be collected and how — has not yet been adopted as of the time of writing. This is an important opportunity: the implementing act could, if drafted with sufficient specificity, effectively create minimum viable execution-time governance requirements without requiring primary legislation. The table below compares what Article 72 currently requires with what it would need to require to constitute meaningful execution-time governance.

Governance Function	Article 72 Current Requirement	What Article 72 Would Need to Require
Runtime Monitoring	Systematic data collection and analysis on performance; basis for a monitoring plan as part of technical documentation	Defined monitoring signals (input drift, output drift, calibration, disaggregated performance); specified statistical methods; quantified alert thresholds; documented alert response protocols
Constraint Enforcement	Not explicitly addressed in Article 72; partially addressed by accuracy and robustness requirements in Chapter III	Documented, versioned constraint specification; requirement that constraint layer changes follow a defined change management process; auditor access rights to constraint documentation
Runtime Attestation	Not addressed	Minimum record content for inference events; cryptographic integrity requirements; retention periods; defined access rights for market surveillance authorities and notified bodies

Intervention Protocol	Incident reporting obligations (Article 73); corrective action requirements	Documented tiered intervention authorization; rollback capability requirement; pre-specified notification obligations by intervention level; mandatory post-intervention review with defined timelines
Continuous Improvement Loop	Post-market monitoring system must support evaluation of continuous compliance	Formal requirement that monitoring findings feed into a governed model review cycle with defined frequency and escalation criteria

8.2 Minimum Viable Execution-Time Governance Requirements

What should a regulatory framework actually specify for high-risk AI systems? We propose that minimum viable execution-time governance requirements for high-risk AI systems include the following. A runtime monitoring specification, submitted as part of conformity assessment documentation, covering the signals monitored, methods used, alert thresholds, and response protocols. A documented constraint specification, covering the constraint layer architecture, the governance process for constraint modifications, and the audit rights of authorized external reviewers. Minimum inference record requirements — what information must be logged for each inference event, or for a statistically defined sample — with tamper-evidence requirements and defined retention periods proportionate to the risk level of the application. An intervention protocol that specifies the authorization chain for each tier of intervention and the notification obligations that attach to each. And an annual governance review requirement that assesses the adequacy of the runtime governance infrastructure against the system's actual operating context, with documented findings.

8.3 Mandated Versus Recommended

We recognize that not all execution-time governance requirements should be mandatory for all high-risk AI systems. A proportionality principle is appropriate: the obligations should scale with the risk level of the system and the volume of its operation. Runtime attestation at the cryptographic level, for example, may be feasible to mandate for high-volume, highest-risk deployments — clinical AI, criminal justice AI, large-scale employment AI — but would impose disproportionate burden on smaller-scale high-risk systems. A sensible regulatory approach would specify mandatory minimum requirements at the lower end of the risk spectrum, with higher-tier requirements either mandated for specific risk categories or achievable through demonstration of equivalence. The standards bodies — particularly ISO/IEC JTC 1/SC 42, which developed ISO 42001, and NIST through the COSAiS project — are well positioned to

develop the technical standards that would allow equivalence demonstrations and provide implementation guidance for organizations that need it.

9. Recommendations

On the basis of this analysis, we make the following seven recommendations, directed specifically at the actors with the authority and the technical capacity to act on them.

Recommendation 1 — For AI Developers: Publish Runtime Governance Specifications as Governance Artifacts. Developers of high-risk AI systems should publish, alongside their model cards and technical documentation, a runtime governance specification that describes the monitoring signals tracked in production, the alert thresholds in use, the constraint layer architecture, and the intervention authorization protocol. This specification should be versioned and updated when the governance architecture changes. Publishing it does not require disclosing proprietary constraint implementations — it requires describing, at a functional level, what governance is in place. This cannot be mandated overnight, but it can begin immediately, and developers who adopt it voluntarily will be better positioned when regulation catches up.

Recommendation 2 — For the European Commission: Use the Article 72 Implementing Act to Establish Minimum Runtime Monitoring Requirements. The Commission has not yet adopted the implementing act specifying the template for post-market monitoring plans under Article 72. This is a concrete legislative opportunity to establish minimum viable execution-time governance requirements without new primary legislation. The implementing act should specify minimum monitoring signals, statistical methods, alert threshold requirements, and documentation standards for high-risk AI systems. It should require that monitoring plans include alert response protocols, not just alert thresholds. It should define minimum inference record requirements proportionate to risk level. The window for this is not indefinite; as the AI Act's high-risk obligations come into full effect in 2026, this implementing act becomes increasingly urgent.

Recommendation 3 — For NIST: Develop an Execution-Time Governance Profile for the AI RMF. The NIST AI RMF profile mechanism — which allows the framework to be adapted for specific use cases and temporal contexts — is exactly the right vehicle for formalizing execution-time governance requirements. NIST should develop a deployment-phase profile that specifies, for high-risk AI systems, the monitoring, constraint, attestation, and intervention governance functions that the framework expects to see in place. This profile should be coordinated with the COSAiS overlays project and with relevant sector-specific initiatives — including the April 2026 concept note on Trustworthy AI in Critical Infrastructure — to ensure that execution-time governance requirements are integrated across NIST's AI governance portfolio. This work can begin with a research note or concept paper within the current fiscal year.

Recommendation 4 — For ISO/IEC JTC 1/SC 42: Develop a Technical Standard for Runtime Attestation of AI Inference Events. The most significant technical gap in the current

execution-time governance landscape is the absence of a standard for runtime attestation. ISO/IEC JTC 1/SC 42 — the subcommittee responsible for ISO 42001 and the broader suite of AI standards — should initiate a work item to develop a technical standard defining minimum content for inference attestation records, integrity requirements, retention guidance, and access control frameworks for high-risk AI deployments. This standard should be coordinated with the confidential computing standards work at the IEEE and with relevant existing standards for audit logging and data integrity. A preliminary technical report within twenty-four months is achievable; a full international standard within forty-eight months is a realistic target.

Recommendation 5 — For Market Surveillance Authorities Under the EU AI Act: Include Runtime Governance Assessment in Post-Market Surveillance Activities. The EU AI Act's market surveillance framework authorizes national market surveillance authorities to request documentation, conduct assessments, and require corrective action. These authorities should develop assessment protocols that specifically evaluate the execution-time governance of high-risk AI systems — not merely whether documentation exists, but whether the runtime monitoring specification is adequate, whether the constraint layer is functioning, and whether intervention protocols are defined and tested. Assessment protocols should be developed collaboratively across Member State authorities to ensure consistency and shared learning.

Recommendation 6 — For AI Deployers in Regulated Domains: Treat Runtime Governance Capability as a Procurement Requirement. Organizations procuring high-risk AI systems for deployment in regulated domains — healthcare providers, financial institutions, public administrations, employment agencies — should establish minimum runtime governance capability as a procurement criterion. This means requiring, as part of vendor selection, that developers provide a runtime governance specification; that monitoring infrastructure is included in the deployment package rather than treated as an optional add-on; and that contract terms specify the monitoring data and attestation records that will be accessible to the deploying organization. Deployers who accept AI systems without runtime governance infrastructure have accepted liability exposure that they may not have fully assessed.

Recommendation 7 — For Research Funders: Prioritize Runtime Attestation Infrastructure and Governance-Compatible Interpretability Research. The two most significant technical gaps supporting execution-time governance — attestation infrastructure that is deployable at production scale, and interpretability methods that are useful for governance rather than primarily for research purposes — are research problems that markets will systematically underinvest in because the benefits are primarily captured by governance actors (regulators, auditors, affected persons) rather than by developers. Research funders — including the European Research Council, UKRI, NSF, and DARPA — should prioritize work on governance-ready attestation infrastructure and on interpretability methods that can support runtime constraint enforcement and audit. This is not a request for basic research funding in a new field. The research base exists. What is needed is targeted investment in making it governance-deployable.

10. Conclusion

The argument of this paper is simple, and we want to close by stating it plainly. We have built a governance architecture for AI systems that is primarily designed to answer one question: is this system safe before it is deployed? That is a real question and it deserves the serious attention it receives. The evaluation frameworks, the conformity assessments, the technical documentation requirements, the benchmark regimes — these represent genuine and valuable governance work. We are not arguing that they are wrong. We are arguing that they are radically incomplete.

The place where AI systems actually cause harm — the place where the consequences of governance success or failure actually materialize — is not design time. It is execution time. It is the moment when a model processes a real input and produces a real output that shapes a real decision affecting a real person. That moment happens billions of times a day, across millions of deployments, in contexts that differ from the evaluation environment in ways that matter enormously and that current governance frameworks have no systematic mechanism to detect. The systems drift. The inputs shift. The confidence scores miscalibrate. The outputs degrade for specific user populations in ways that the aggregate metrics never surface. And there is, for most of these systems in most of these deployments, no governance infrastructure watching.

We are governing AI systems the way we govern bridges — inspecting the blueprints and testing the materials, but not the bridge while people are crossing it. For a bridge, this is defensible: the bridge's behavior is fixed at construction, and a properly constructed bridge that behaves well in post-construction testing will behave well for its operational lifetime, short of extraordinary events. For AI systems, it is not defensible. The behavior is not fixed. It is a function of runtime inputs that change continuously. The evaluation tells you how the system behaved on a test set at a specific point in time. It tells you nothing about how it will behave on your users next month, or what it will do when the input distribution shifts, or whether the confidence scores it is generating are still reliable guides to when human review is warranted.

Closing Argument

Design-time governance is necessary but nowhere near sufficient for AI systems that produce consequential outputs at scale over time. The governance architecture we have built for a model of technology — fixed at manufacture, predictable within tested parameters — that does not describe AI systems. Execution-time governance is not an advanced feature for a future generation of frameworks. It is the missing layer that determines whether the governance we have already built actually protects anyone. Building it requires specific technical infrastructure, specific regulatory requirements, and specific institutional capacity. All of these are achievable. None of them will happen spontaneously. The question is only whether they happen before or after the harms become undeniable.

The five challenges we have identified — opacity, scale, latency, adversarial robustness, and jurisdiction — are real, and they will require serious engineering and legal work to navigate. None of them is a reason to defer. The technical infrastructure for runtime attestation is further

along than most governance discussions acknowledge. The monitoring tools are capable, even if their governance integration is immature. The regulatory frameworks have leverage points — Article 72's implementing act, NIST's profile mechanism, ISO's standards pipeline — that can be used now, within existing legislative authority, to begin building the requirements that the market will not build without them.

What is missing is not, primarily, technical capability. What is missing is the governance decision to treat AI system behavior in production as an object of governance, not merely a consequence of design-time choices that governance has already addressed. That decision is ours to make. The system is running. The question is whether anyone is governing it — and right now, for most systems in most deployments, the answer is no.

Appendix: Design-Time vs. Execution-Time Governance Obligations Across Major Frameworks

Framework	Key Design-Time Obligations	Key Execution-Time Obligations	Execution-Time Gaps
EU AI Act (2024/1689)	Conformity assessment; technical documentation; risk management system; data governance; accuracy and robustness requirements; transparency; EU database registration; quality management system	Post-market monitoring plan (Art. 72); serious incident reporting (Art. 73); corrective action obligations; market surveillance cooperation	No mandatory monitoring signal specification; no alert threshold requirements; no inference record requirements; no constraint layer audit rights; no intervention authorization framework; implementing act not yet adopted
NIST AI RMF 1.0 (2023)	GOVERN: organizational accountability structure; MAP: context and risk categorization; MEASURE: evaluation,	MANAGE function includes monitoring as ongoing activity; real-time monitoring mentioned as a safety practice; IMPROVE	Voluntary and non-prescriptive; no mandatory monitoring specifications; no attestation requirements; no defined intervention

	testing, benchmarking; MANAGE: risk treatment planning	function addresses feedback loops	governance; deployment-phase profile not yet developed
ISO 42001:2023	AI management system requirements; AI risk assessment and treatment; AI system impact assessment; operational planning and control; competence and awareness requirements	Monitoring, measurement, analysis and evaluation (Clause 9.1); internal audit; management review; nonconformity and corrective action	Management system standard focused on organizational processes; does not specify technical runtime governance requirements; monitoring requirements are organizational level, not inference level; no attestation standard
NIST SP 800-53 Rev. 5	System and services acquisition controls; configuration management; risk assessment; planning; program management	Audit and accountability (AU family); system and information integrity (SI family including SI-4 system monitoring); incident response (IR family); assessment and authorization monitoring (CA-7)	Not AI-specific; COSAiS project (2025–2026) developing AI-specific overlays; current controls provide useful framework but require significant adaptation for AI inference governance
IEC 62443 (Partial Analogy)	Secure product development lifecycle; security level requirements; system architecture; component security requirements; supply chain security	Security management (2-x); runtime monitoring of OT environments; incident response; change management for control logic	Designed for deterministic OT systems; does not address probabilistic AI behavior; AI-specific extensions under community discussion but not yet standardized; zones-and-conduits model partially applicable to AI

			deployment architectures
--	--	--	--------------------------

References

- [1] European Parliament and Council of the European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union. OJ L, 2024/1689.
- [2] National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.100-1>
- [3] International Organization for Standardization. (2023). *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*. ISO/IEC.
- [4] National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations: NIST Special Publication 800-53, Revision 5*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>
- [5] National Institute of Standards and Technology. (2025–2026). *Control Overlays for Securing AI Systems (COSaIS) — Concept Paper and Annotated Outline*. NIST Computer Security Resource Center. <https://csrc.nist.gov/projects/sp-800-53-control-overlays-for-securing-ai-systems>
- [6] National Institute of Standards and Technology. (2024). *NIST-AI-600-1: Artificial Intelligence Risk Management Framework — Generative Artificial Intelligence Profile*. U.S. Department of Commerce.
- [7] Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., ... & Kaplan, J. (2022). *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. Anthropic. arXiv:2204.05862.
- [8] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., ... & Clark, J. (2022). *Constitutional AI: Harmlessness from AI Feedback*. Anthropic. arXiv:2212.08073.
- [9] Kundu, S., Bai, Y., Kadavath, S., Askell, A., Callahan, A., Chen, A., ... & Kaplan, J. (2023). *Specific versus General Principles for Constitutional AI*. Anthropic. arXiv:2310.13798.
- [10] Schnabl, C., Hugenholtz, D., Marino, B., & Raad, A. (2025). *Attestable Audits: Verifiable AI Safety Benchmarks Using Trusted Execution Environments*. arXiv:2506.23706.
- [11] Woodworth, J., & Yao, Z. (2024). *Confidential Neural Computing: Hosting Generative AI Model Workloads in a Trusted Execution Environment*. Google Research.
- [12] Chen, B., Dong, C., Wang, G., Shen, Y., & Li, Y. (2024). *Securing AI Inference in the Cloud: Is CPU-GPU Confidential Computing Ready?* IEEE Cloud Computing 2024. <https://doi.org/10.1109/CLOUD62652.2024>
- [13] Shao, M., Li, D., Zhao, C., Wu, X., Lin, Y., & Tian, Q. (2024). *Supervised Algorithmic Fairness in Distribution Shifts: A Survey*. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI 2024), pp. 8225–8233. <https://doi.org/10.24963/ijcai.2024/909>
- [14] Arize AI. (2024). *Arize AI Platform Documentation: Drift Detection, Bias Tracing, and Model Observability*. Arize AI, Inc. <https://docs.arize.com>
- [15] Fiddler AI. (2024). *Fiddler AI Platform: Explainability, Fairness, and Trust Service Documentation*. Fiddler AI, Inc. <https://docs.fiddler.ai>

16. [16] Evidently AI. (2024). *Evidently AI: Open-Source ML Monitoring and Testing Library Documentation*. Evidently AI. <https://docs.evidentlyai.com>
17. [17] Databricks. (2024). *MLflow Documentation: Model Tracking, Versioning, and Production Monitoring*. Databricks, Inc. <https://mlflow.org/docs/latest/index.html>
18. [18] Weights & Biases. (2024). *Weights & Biases: ML Experiment Tracking and Model Monitoring Documentation*. Weights & Biases, Inc. <https://docs.wandb.ai>
19. [19] International Society of Automation / International Electrotechnical Commission. (2018). *IEC 62443: Industrial Automation and Control Systems Security*. ISA/IEC.
20. [20] Weiss, E. (2026, May 13). Healthcare AI Has an Accountability Problem. *Unite.AI*. <https://www.unite.ai/healthcare-ai-accountability-problem/>
21. [21] International Association of Privacy Professionals. (2026, May 20). Governing AI That Actually Listens. *IAPP News & Analysis*. <https://iapp.org/news/a/governing-ai-that-actually-listens>
22. [22] NFM Consulting. (2026, April 2). OT AI Governance and Cybersecurity — Managing Risk When AI Agents Enter the Control System. *NFM Consulting Knowledge Base*.
23. [23] National Institute of Standards and Technology. (2026, April 7). *AI RMF Profile on Trustworthy AI in Critical Infrastructure — Concept Note*. NIST AI Resource Center.
24. [24] AI Act Service Desk. (2024). *Article 72: Post-Market Monitoring by Providers and Post-Market Monitoring Plan for High-Risk AI Systems — Summary and Guidance*. European Commission AI Act Service Desk. <https://ai-act-service-desk.ec.europa.eu>
25. [25] Goodfellow, I., McDaniel, P., & Papernot, N. (2018). Making machine learning robust against adversarial inputs. *Communications of the ACM*, 61(7), 56–66. <https://doi.org/10.1145/3134599>
26. [26] Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
27. [27] Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), 1–35. <https://doi.org/10.1145/3457607>
28. [28] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
29. [29] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*.
30. [30] Seshia, S. A., Sadigh, D., & Sastry, S. S. (2022). Toward verified artificial intelligence. *Communications of the ACM*, 65(7), 46–55. <https://doi.org/10.1145/3503914>

© 2026 Hollow House Institute. This work is distributed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).